

Capture the Flag 5: Make the System Interactive

🚩 Flag 0 — Create Your Feature Branch

Objective: Create a new feature branch before adding interactions.

Create:

`feature/student-interactions`

Screenshot:

- Terminal showing your current branch

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\leila\Documents> git branch
  feature/data-ui
* feature/student-list
  feature/widgets
  master
PS C:\Users\leila\Documents> git checkout -b feature/student-interactions
Switched to a new branch 'feature/student-interactions'
PS C:\Users\leila\Documents> git branch
  feature/data-ui
* feature/student-interactions
  feature/student-list
  feature/widgets
  master
PS C:\Users\leila\Documents>
```

🚩 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using **onPressed**.

Right now your buttons are only decorations.

Research how Flutter's **onPressed** works.

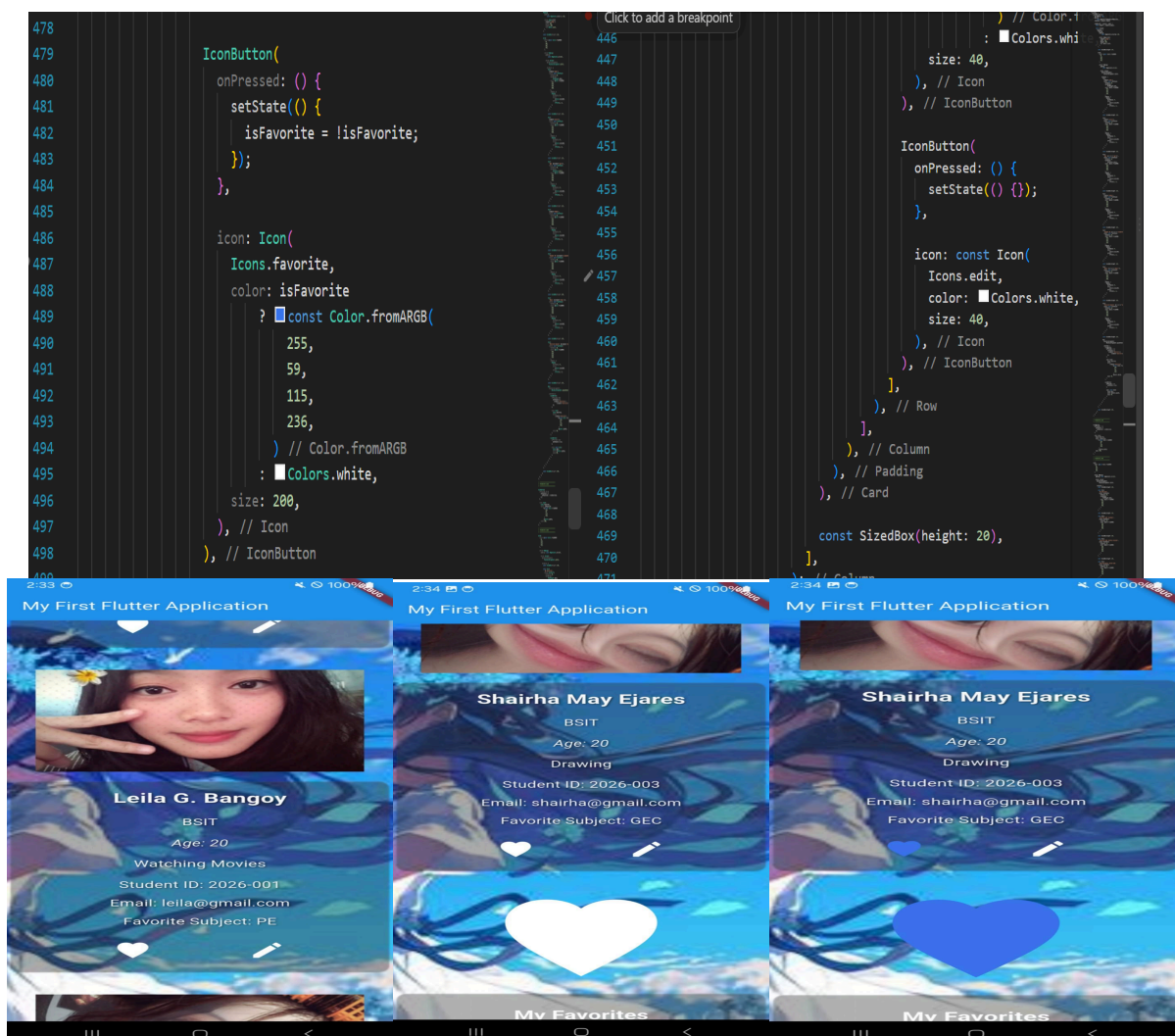
The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

Test

When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

Screenshot:

- Student Card with buttons
- **onPressed** inside your code
- Button reacting or terminal message



🚩 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using `onTap`.

Buttons aren't the only interactive elements.

Research Flutter's `onTap`.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

Test

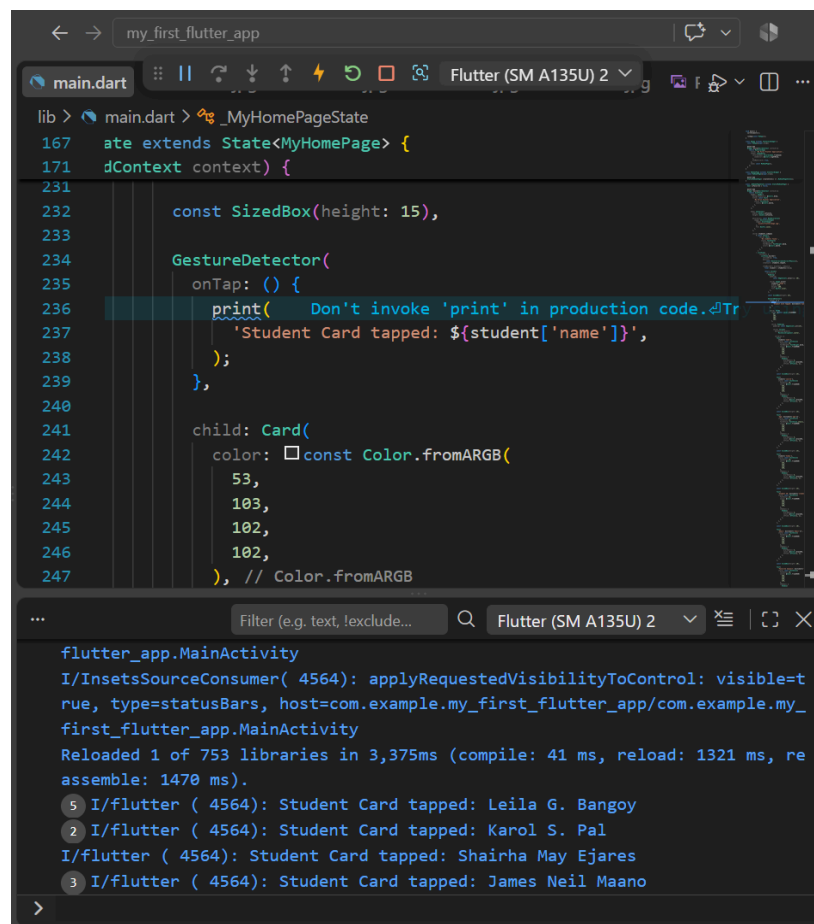
The user should be able to:

- Tap the Favorite button
- Tap the Edit button
- Tap anywhere else on the Student Card

Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

Screenshot:

- `onTap` in your code
- Student Card being tapped



```
lib > main.dart > _MyHomePageState
167   ate extends State<MyHomePage> {
171   dContext context) {
231
232     const SizedBox(height: 15),
233
234     GestureDetector(
235       onTap: () {
236         print(' Don't invoke 'print' in production code. Tr
237           'Student Card tapped: ${student['name']}',
238         );
239       },
240
241       child: Card(
242         color: const Color.fromARGB(
243           53,
244           103,
245           102,
246           102,
247         ), // Color.fromARGB

...
Filter (e.g. text, !exclude...
Flutter (SM A135U) 2

flutter_app.MainActivity
I/InsetsSourceConsumer( 4564): applyRequestedVisibilityToControl: visible=true, type=statusBars, host=com.example.my_first_flutter_app/com.example.my_first_flutter_app.MainActivity
Reloaded 1 of 753 libraries in 3,375ms (compile: 41 ms, reload: 1321 ms, reassemble: 1470 ms).
5 I/flutter ( 4564): Student Card tapped: Leila G. Bangoy
2 I/flutter ( 4564): Student Card tapped: Karol S. Pal
I/flutter ( 4564): Student Card tapped: Shairha May Ejares
3 I/flutter ( 4564): Student Card tapped: James Neil Maano
```

🚩 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

The important part is understanding that `setState` tells Flutter to rebuild the UI.

Test

The screen should visibly change immediately after pressing the button.

Screenshot:

- `setState` in your code
- Before and after interaction



🚩 Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

Each student's favorite status should work independently.

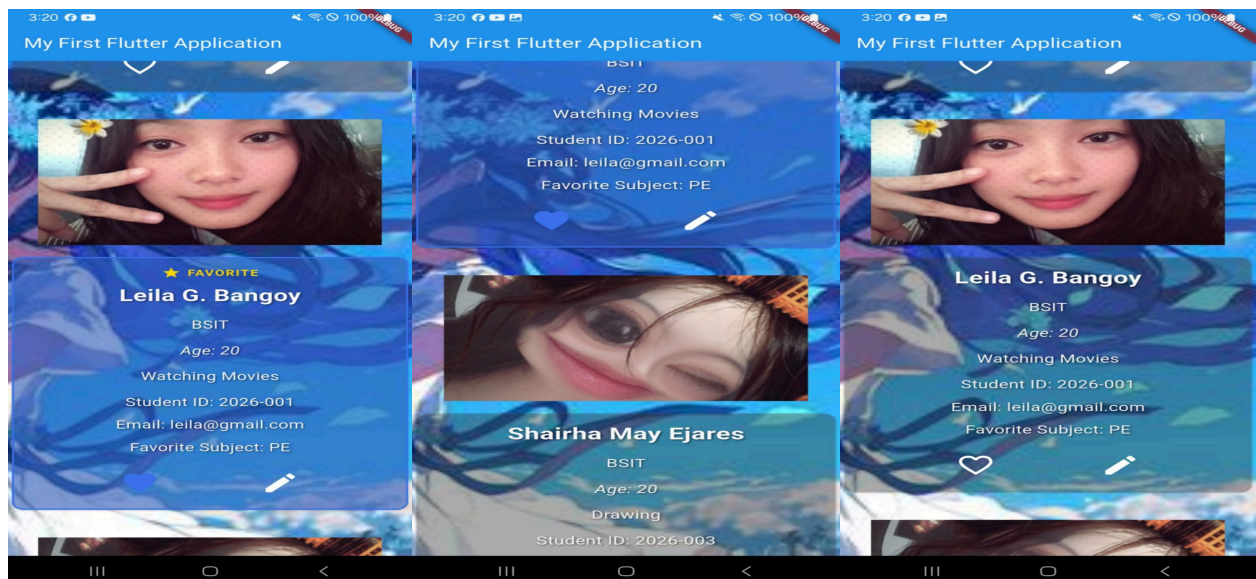
Test

- Favorite one student.
- Other students should remain unchanged.
- Favorite another student.

Both should keep their own state.

Screenshot:

- Favorited student
- Non-favorited student
- Favorite icon changing



🚩 Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart [List](#).

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

Test

Before:

- Student A
- Student B
- Student C

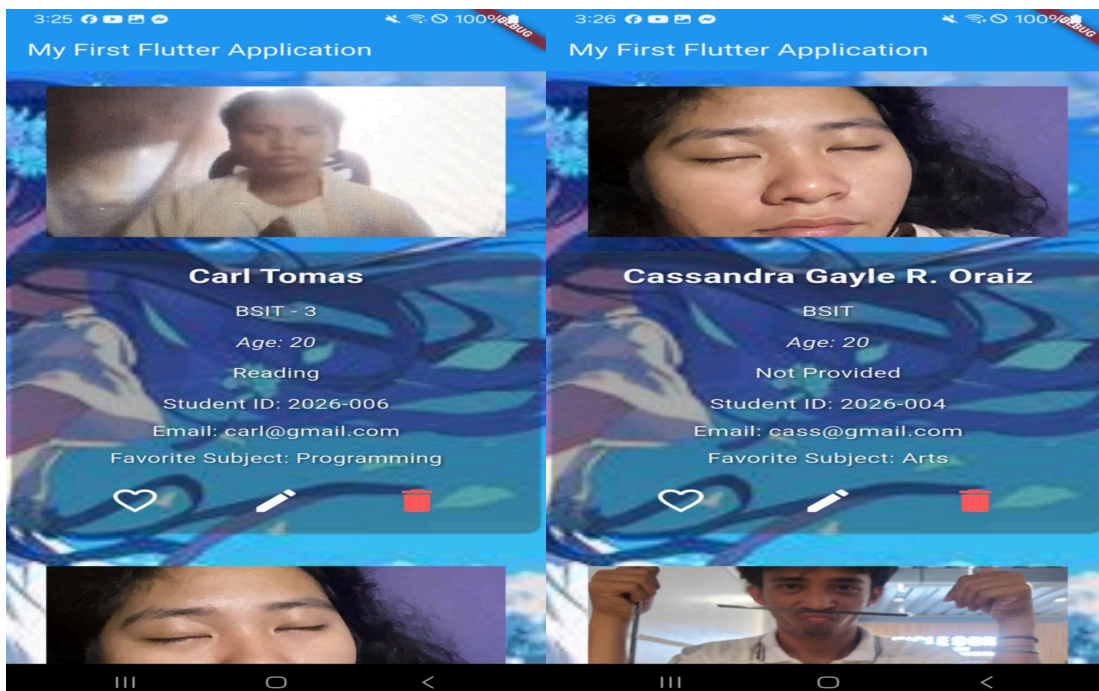
After deleting Student B:

- Student A
- Student C

No empty spaces should remain.

Screenshot:

- Student before deletion
- Student after deletion



🚩 Flag 6 — Edit Trigger

Objective: Prepare your app for editing student information.

In the next CTF you'll build a full Edit Screen.

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

- `AlertDialog`
- `SnackBar`
- Bottom Sheet

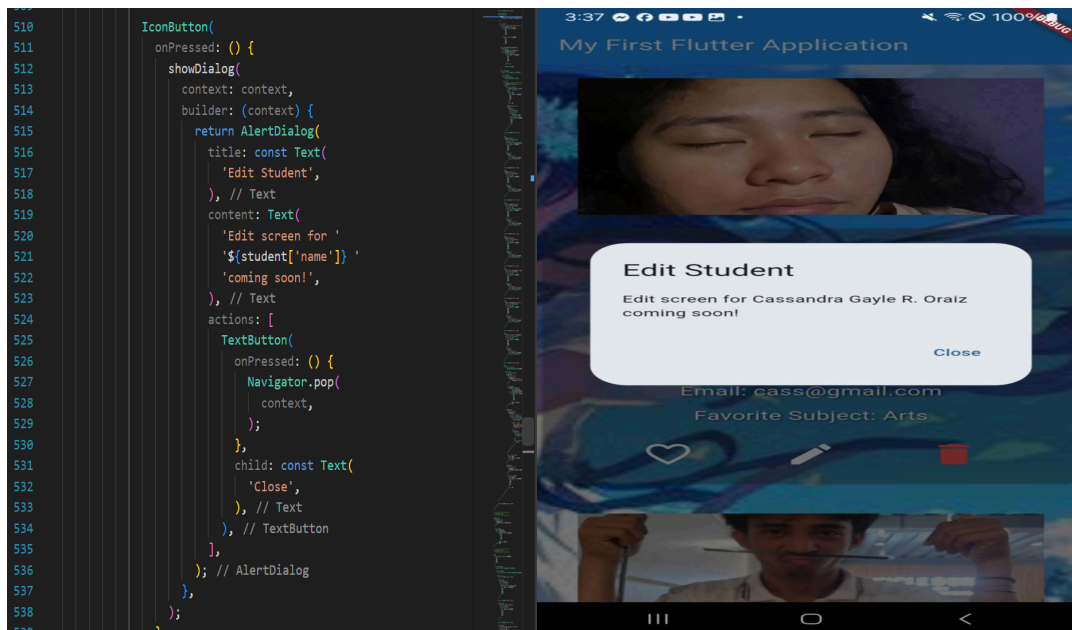
The goal is simply to launch an edit action, not build the complete editor yet.

Test

Pressing Edit should open your chosen interface.

Screenshot:

- Edit button
- Opened dialog, snackbar, or bottom sheet



🚩 Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

This is your final interaction challenge.

Verify that every student behaves independently.

Example scenario:

- Favorite Student A.
- Delete Student C.
- Open Edit for Student B.

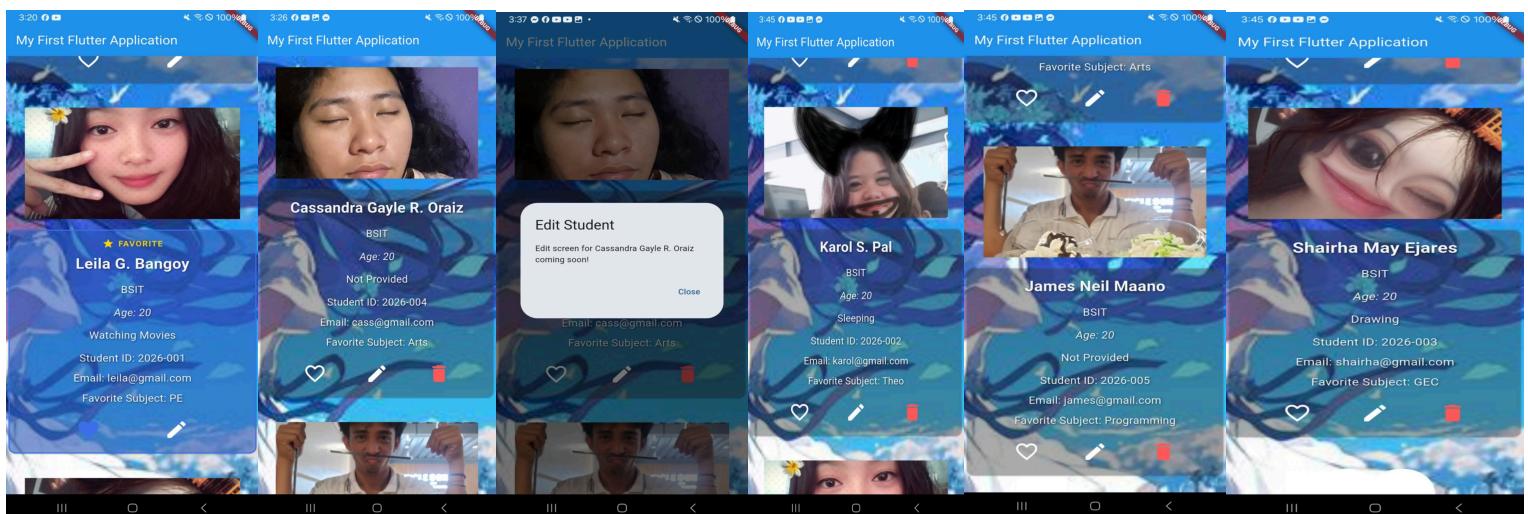
Your application should continue functioning correctly without affecting unrelated students.

Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

Screenshot:

- Multiple interactions working together
- Application still functioning normally



Leila - As Favorite, Joshua - Deleted from student Display card list, Cass - in Edit mode, And the 3 remainings in there states.

🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

Branch:

`feature/student-interactions`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
C:\Users\leila\Documents>git commit -m "Feat: Added updated Activity 5"
[feature/student-interactions 595ba0c] Feat: Added updated Activity 5
1 file changed, 345 insertions(+), 188 deletions(-)

C:\Users\leila\Documents>git branch
  feature/data-ui
* feature/student-interactions
  feature/student-list
  feature/widgets
  master

C:\Users\leila\Documents>git push origin feature/student-interactions
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 2.03 KiB | 346.00 KiB/s, done.
Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote:
remote: Create a pull request for 'feature/student-interactions' on GitHub by visiting:
remote:   https://github.com/leilabangoy22/IT314AB_BANGOY/pull/new/feature/student-interactions
remote:
To https://github.com/leilabangoy22/IT314AB_BANGOY.git
 * [new branch]      feature/student-interactions -> feature/student-interactions
```